ШКОЛСКИ КОМПАЈЛЕР

- Структура
- Основне фазе компајлирања

Дошли смо до пола пута

- Знамо како да направимо програм који разуме улазни код.
- Школски компајлер је довршетак тог пута са што мањим напором.

Школски компајлер

- Школски компајлер преводи програме написане у Праволинијском програмском језику на асемблерски језик ЛПРС процесора.
- У 23 датотеке и испод 1500 линија кода.
- Нека ограничења:
  - Нема гранања (улазни језик је такав)
  - Слаба детекција грешака у улазној датотеци и опоравак од њих
  - Нема ослобађања динамички заузетих објеката
  - Постоји само један тип у улазном језику – целобројни
  - Нема провере семантике
  - Генерише се асемблер без сегмената
  - Постоји само један досег (и нема функција)
  - Сви напосредни операнди могу заузимати највише 6 бита
  - ...

Структура школског компајлера

- Ово су фазе у школском компајлеру:
- **лексичка** анализа
- **синтаксна** анализа (парсинг)
- **генерисање** инструкција
- **придруживање** меморије
- **асемблирање**.

Синтакса праволинијског језика

Stm → ***Stm*** ; ***Stm***	(CompoundStm)	***ExpList*** → ***Exp*** , ***ExpList***	(PairExpList)
Stm → id := ***Exp***	(AssignStm)	***ExpList*** → ***Exp***	(LastExpList)
Stm → print (***ExpList***)	(Printstm)	***Binop*** → +	(Plus)
Exp → id	(IdExp)	***Binop*** → -	(Minus)
Exp → num	(NumExp)	***Binop*** → x	(Times)
Exp → ***Exp Binop Exp***	(OpExp)	***Binop*** → /	(Div)
Exp → (***Stm*** , ***Exp***)	(EseqExp)		

- Школски компајлер не подржава множење и дељење.
- print наредба се своди на уписивање вредности на адресу 31

Лексичка анализа

- Лексичка анализа се не ослања детерминистички аутомат са коначним бројем стања, већ мало спорију технику, али релативно једноставнију за ручну имплементацију.
- Лексер нуди функције:
  - добави наредни токен
  - врати назад последњи токен
  - вирни у наредни токен (без добављања)

Синтаксна анализа

- Имплементиран коришћењем алгоритма са рекурзивним спуштањем.
- Уочите како је граматика промењена, да не би имала конфликте.
- Резултат парсирања је стабло синтаксне анализе (енгл. AST - Abstract syntax tree) које је по структури идентично са другим примером међурепрезентације који смо дали.
- Постоји и модул за штампање AST-а, који можете користити за бољи увид у програм и потврду да је улаз добро синтаксно анализиран.

Семантичка анализа

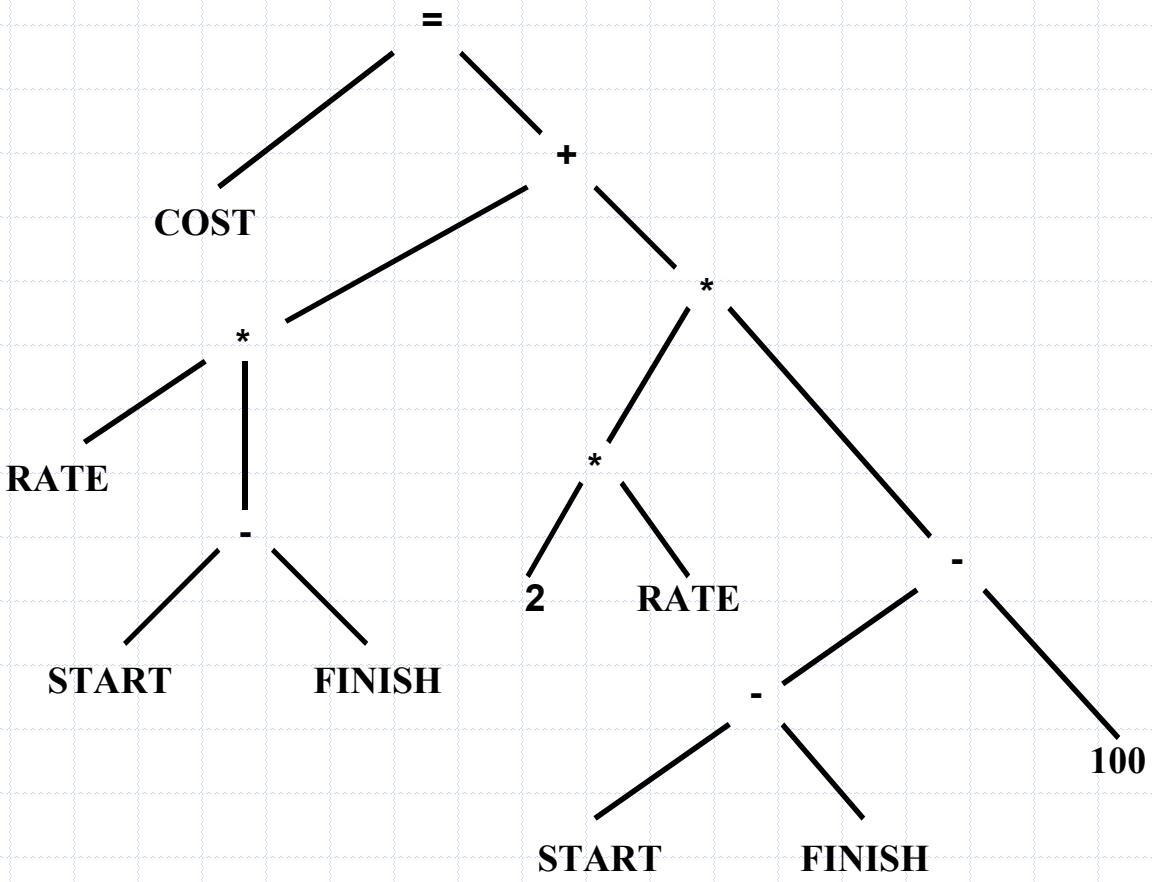
- У школском компајлеру се не обавља семантичка анализа.
- Какве све семантичке грешке би се могле хватати у оваквом програму?
- Какве семантичке грешке постоје у неким другим програмским језицима?

Прелазни облици полазног програма:

- Прелазни облик полазног програма (илити „међурепрезентација“) која се добија на излазу синтаксне анализе је у школском компајлеру стабло синтаксне анализе.
- Уместо стабла, могао би се направити тзв. „матрични прелазни облик“

Пример стабла синтаксне анализе:

COST = RATE ∗ (START - FINISH) + 2 ∗ RATE ∗ (START - FINISH - 100)



Пример матричног прелазног облика:

VRSTA U MATRICI	OPERATOR	OPERAND 1	OPERAND 2
1	-	START	FINISH
2	*	RATE	M_1
3	*	2	RATE
4	-	START	FINISH
5	-	M_4	100
6	*	M_3	M_5
7	+	M_2	M_6
8	=	COST	M_7

Оптимизација

- У школском компајлеру се не обављају никакве оптимизације
- Бројне технике постоје.
- Две категорије: Машински независне и машински зависне оптимизације.
- Примери машински независних: елиминација заједничких подизраза, претходно израчунавање вредности, уочавање инваријантног рачунања унутар циклуса...
- Примери машински зависних: уклањање вишка LOAD и STORE, JUMP на JUMP, итд.

Пример оптимизације: претходно израчунавање вредности у исказу A = 2 * 276/92 * B

PRE OPTIMIZACIJE			
M_1	*	2	276
M_2	/	M_1	92
M_3	*	M_2	B
M_4	=	A	M_3
POSLE OPTIMIZACIJE			
M_1			
M_2			
M_3	*	6	B
M_4	=	A	M_3

Генерисање инструкција

- Фаза генерисања (или: избор) инструкција обавља превођење операција прелазног облика у операције (инструкције) одредишног језика.
- Конкретна изведба зависи од форме прелазног облика.
- Код матричне форме, иде се редом по врстама и за сваку врсту се генерише одговарајући скуп инструкција. Слично макро асемблеру.
- Код стабла, пролази се кроз стабло и почевши од листова се за сваки чвор генерише одговарајући скуп инструкција.

Примери смена (макродефиниција):

SUBM	MACRO	MI, OPRND1, OPRND2
	MOV	AX, OPRND1
	SUB	AX, OPRND2
	MOV	MI, AX
	ENDM	
ADDM	MACRO	MI, OPRND1, OPRND2
	MOV	AX, OPRND1
	ADD	AX, OPRND2
	MOV	MI, AX
	ENDM	

Примери смена - наставак:

```
MULM    MACRO   MI, OPRND1, OPRND2
        MOV     AX, OPRND1
        MUL     AX, OPRND2
        MOV     MI, AX
        ENDM
ASNM    MACRO   OPRND1, OPRND2
        MOV     AX, OPRND2
        MOV     OPRND1, AX
        ENDM
```

Придруживање меморије:

- Додела меморије свим променљивама
- али и свим привременим операндима које су потребне за међурезултате (настају приликом прављења матричног облика, или приликом генерисања инструкција),
- Меморија се може доделити и литералима (непосредним операндима)
- Претпоставља се да је меморија довољно велика.

Асемблирање

- Уколико се у фази генерисања кода произведе асемблерски код, потребно је користити и асемблер ради добијања потребног кода на машинском језику.
- Дакле, да би се добио машински код, треба излаз из школског компајлера проследити асемблерском преводиоцу за ЛПРС процесор. (Да ли бисте умели сами да га направите?)

Задаци за размишљање

- Пробајте да разумете како је измењена граматика да би се могао применити алгоритам са рекурзивним спуштање. (Уочите где се користи FOLLOW скуп.)
- Размислите шта би требало да се уради да би школски компајлер подржао множење и дељење.
- Како увести подршку за непосредне операнде до 15 бита?
- Шта треба изменити да би се ухватила (семантичка) грешка када се променљива користи пре доделе вредности?
- Како би имплементирали оптимизацију срачунавања израза?
- Размислите о стављању непосредних операнада (литерала) у меморију. Шта се тиме добија (а шта губи)?
- Како би преправили код да користи AST имплементиран на неки од осталих начина за које смо дали код (са полиморфизмом, обрасцем посетиоца, std::variant...)?